SAE M2 DFS & DE

Génération des emplois du temps

Groupe 3 — Projet B, modélisation et historique des algorithmes

Version de travail — novembre 2026

---

1 Le problème

Entrée, pour une semaine : les séances à placer (matière, type CM/TD/TP, profs, groupes, durée de 1 h ou 1 h 30, type de salle requis), les salles (capacité, type), la hiérarchie des groupes (promo → TD → TP), les indisponibilités des profs et des salles, les préférences des profs.

Sortie : pour chaque séance, un jour, une heure de début et une salle. Une séance impossible à placer est laissée de côté plutôt que de casser une règle.

Grille : du lundi au vendredi, 8 h 30–12 h et 13 h 30–18 h 30, au pas de 30 min. Un cours tient entièrement dans une demi-journée, soit 15 débuts possibles par jour pour 1 h et 13 pour 1 h 30. Le semestre est généré semaine par semaine.

2 Contraintes et score

Contrainte	**Définition**	**Coût**
Dures : une seule violation rend la solution invalide		
Prof	un prof n'a jamais deux cours en même temps	—
Groupe	deux cours simultanés n'ont pas d'étudiants en commun : un CM de promo bloque tous ses TD et TP, mais TD2 et un TP du TD1 peuvent avoir lieu en même temps	—
Salle	une salle n'accueille qu'un cours à la fois, assez grande et du bon type (salle de TP)	—
Indisponibilités	prof ou salle indisponible	—
Souples : mesurées par un score, à minimiser		
Trou	temps libre entre deux cours d'une même demi-journée, vu par chaque groupe le plus fin avec les cours de ses groupes parents ; la pause de midi n'en est pas un	1 si < 1 h, sinon 3 / h
Préférence	cours d'un prof en dehors de ses jours ou demi-journées préférés	1
Équilibre	écart entre les heures de cours d'un jour et la moyenne de la semaine	1 / h

Les trous ont le poids le plus fort : c'est le critère de confort le plus visible pour les étudiants.

Complexité. Si l'on ne garde que les conflits entre séances, placer n séances sur k créneaux revient à colorier un graphe avec k couleurs, problème NP-complet. L'optimum est donc hors d'atteinte sur de grandes instances : on utilise des heuristiques et on les compare.

3 Méthode de comparaison

- **Mêmes données** : chaque instance est stockée en JSON (`backend/scheduling/instances/`) ; tous les algorithmes tournent sur les mêmes fichiers.
- **Même juge** : un évaluateur unique compte les violations par type, les séances non placées et le score souple.
- **Classement** : d'abord le nombre de violations, puis les séances non placées, puis le score souple. Comparer les scores souples n'a de sens qu'entre solutions valides.
- **Hasard** : les algorithmes aléatoires tournent avec plusieurs graines (moyenne et écart-type) ; chaque résultat garde le commit qui l'a produit.
- **Historique** : une nouvelle version s'ajoute à côté de l'ancienne (`greedy@1`, `greedy@2`…), qui reste mesurée.

Commande : `python -m scheduling.benchmark` (depuis `backend/`).

4 Historique des versions

Version	Principe	Complexité	Limites
`random@1` (référence)	Heure et salle tirées au hasard pour chaque séance, sans aucune contrainte. Sert de plancher à battre.	$O(n)$	Presque jamais valide.
`greedy@1`	Séances les plus contraintes d'abord (moins de salles possibles, puis plus de conflits, puis plus longues) ; chacune prend le premier créneau libre de la semaine et la plus petite salle qui convient ; aucun retour arrière.	$O(n \log n)$ pour le tri, puis $O(n \cdot S \cdot (R+n))$ au pire (S débuts possibles, R salles)	Ne revient jamais sur un choix : peut laisser des séances non placées alors qu'une solution existe. Remplit le début de semaine en premier.
`dsatur@1` *à venir*	Coloration du graphe des conflits (DSatur), puis attribution des salles créneau par créneau.	$O(n^2)$ pour la coloration	—
`annealing@1` *à venir*	Recuit simulé à partir de la solution DSatur, pour réduire les trous et le déséquilibre.	O(itérations × coût d'un mouvement)	—

4.1 Exemple : le glouton sur la petite instance

Instance de test `tiny` : une promo (60 étudiants) avec TD1, TD2 et un groupe de TP sous TD1 ; deux profs, dont Bob indisponible le lundi matin ; un amphi, une salle de 35 places, une salle de TP.

Lundi	Cours	Salle
8 h 30 – 10 h	CM promo (Alice)	amphi
10 h – 11 h 30	TP du TD1 (Alice)	salle de TP
13 h 30 – 15 h	TD1 (Bob)	salle 35 places
15 h – 16 h	TD2 (Bob)	salle 35 places

Toutes les contraintes dures sont respectées et il n'y a aucun trou, mais **tout est placé le lundi** : le glouton prend toujours le premier créneau libre. C'est la faiblesse que les versions suivantes devront corriger.

5 Résultats

Instance	Algorithme	Exécutions	Valides	Violations	Non placées	Score souple
`tiny`	`random@1`	5	0 %	2,8	0	9,68 ± 1,47
`tiny`	`greedy@1`	1	100 %	0	0	11,20

Le hasard obtient un meilleur score souple parce qu'il étale les cours sur la semaine, mais ses solutions sont invalides : il est classé derrière. Ces résultats ne portent que sur une instance de 4 séances ; les jeux de données de tailles croissantes (petit, moyen, grand) viendront du générateur.

6 Questions ouvertes

- Le poids de l'équilibre (8,4 points pour le glouton sur `tiny`) risque d'écraser celui des trous sur de petites semaines : à revoir avec les résultats sur les grandes instances.
- Le glouton gagnerait-il à choisir le créneau le moins chargé plutôt que le premier ? Ce serait `greedy@2`.